



UNIVERSIDADE
FEDERAL DO CEARÁ

Universidade Federal do Ceará

Residência em segurança cibernética

CA00068 - Segurança em aplicações web

Professor: Emanuel Bezerra Rodrigues

Análise de segurança em aplicações web

*SAST, SCA e DAST aplicados à identificação, classificação e
exploração de vulnerabilidades*

Terça, 11 de Agosto de 2026
Isis Nascimento de Lavor - 607756

Sumário

1	Introdução	1
2	Análise <i>SAST</i> - <i>Semgrep</i>	1
2.1	Vulnerabilidade: <i>Unencrypted request over HTTP detected</i>	1
2.2	Vulnerabilidade: <i>OS Command Injection</i>	2
2.3	Resumo comparativo das vulnerabilidades	3
3	Análise <i>SCA</i> - <i>OSV Scanner</i>	3
3.1	CVE-2025-7783: <i>Form-data</i>	3
3.2	CVE-2026-3635: <i>Fastify</i>	4
4	Análise <i>DAST</i> - <i>Burp Suite</i>	4
4.1	Desafio e configuração do ambiente	4
4.2	Análise e exploração da vulnerabilidade	6
4.3	Evidência e obtenção de flag	8
4.4	Recomendações sobre a vulnerabilidade	10
5	Conclusão	10
A	Apêndice: <i>SAST</i> com <i>Semgrep</i>	13
B	Apêndice: <i>SCA</i> com <i>OSV Scanner</i>	14

Lista de Figuras

1	Tela do navegador interno do Burp Suite.	5
2	Desafio escolhido para realização da atividade.	5
3	Tela inicial de proxy do Burp com visualização de requisições	6
4	Primeira inserção de payload para teste.	7
5	Visualização de orderby com tabela maior do que nosso valor desejado.	8
6	Payload que gera a resolução final do desafio.	9
7	Resolução final do desafio na plataforma.	10
8	Terminal com comandos de análise semgrep.	13
9	Terminal com comandos para transformar os resultados em formato HTML.	13
10	Visualização dos dados em site gerado pela ferramenta.	14
11	Terminal com comando utilizados para análise com OSV.	14
12	Resultado da análise.	15

Lista de Tabelas

1	Resumo das vulnerabilidades analisadas	3
---	--	---

1 Introdução

A identificação e o tratamento de vulnerabilidades constituem etapas fundamentais do processo de desenvolvimento e manutenção de aplicações seguras. Nesse contexto, diferentes técnicas de análise podem ser empregadas de forma complementar, permitindo examinar tanto o código-fonte e os componentes utilizados por uma aplicação quanto seu comportamento durante a execução. Este relatório apresenta a realização de análises de segurança por meio de três abordagens: *Static Application Security Testing* (SAST), *Software Composition Analysis* (SCA) e *Dynamic Application Security Testing* (DAST). A análise SAST foi conduzida com a ferramenta Semgrep, com o objetivo de identificar potenciais vulnerabilidades diretamente no código-fonte da aplicação *Broken Crystals*. Para a análise SCA, foi utilizado o *OSV Scanner*, destinado à identificação de vulnerabilidades conhecidas nos componentes e dependências empregados pela aplicação.

Por fim, a abordagem DAST foi aplicada mediante a exploração manual de um desafio da categoria *Web* da plataforma *Forge RSI*, utilizando o *Burp Suite* como ferramenta de apoio à análise das requisições e respostas da aplicação. Além da identificação das vulnerabilidades, as análises buscam compreender suas características e possíveis impactos, relacionando-as, quando aplicável, às classificações *Common Weakness Enumeration* (CWE) e OWASP Top 10 2025, bem como avaliar sua criticidade e as medidas adequadas para mitigação ou remediação. Dessa forma, o relatório documenta os procedimentos realizados e apresenta uma análise técnica dos principais achados obtidos em cada uma das abordagens utilizadas.

2 Análise SAST - Semgrep

Nesta seção são apresentadas as respostas às questões propostas, com ênfase na realização das análises no *Semgrep*. As capturas de tela utilizadas para documentar as etapas e os resultados obtidos encontram-se disponíveis no Apêndice A, ao final deste relatório.

2.1 Vulnerabilidade: *Unencrypted request over HTTP detected*

O *Semgrep* identificou no arquivo `brokencrystals/src/email/email.service.ts` uma requisição realizada por *HTTP* sem proteção criptográfica, classificando o achado como **High** e associando-o ao *CWE-319 – Cleartext Transmission of Sensitive Information* e à categoria *A04:2025 – Cryptographic Failures do OWASP Top 10*. O *CWE-319* ocorre quando informações sensíveis ou relevantes para segurança são transmitidas em texto claro por um canal suscetível à interceptação. No código analisado, o serviço de e-mail utiliza uma URL iniciada por `http://` para acessar o *MailCatcher* e recuperar mensagens e seus respectivos conteúdos, possibilitando que informações trafegadas sejam observadas caso um atacante consiga monitorar o canal de comunicação.

A vulnerabilidade enquadra-se em *A04:2025 – Cryptographic Failures* porque essa categoria compreende falhas decorrentes da ausência ou do emprego inadequado de mecanismos criptográficos, incluindo a transmissão de dados sem proteção adequada. Como medida de correção, recomenda-se substituir a comunicação HTTP por HTTPS utilizando TLS adequadamente configurado e realizar a validação correta dos certificados. Utilizando o CVSS 4.0, foi definido o vetor `CVSS:4.0/AV:A/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`, resultando em *score* **7.1**, correspondente à criticidade **High**. Considerando esse nível de severidade, sugere-se um prazo máximo de 15 dias para remediação, priorizando a proteção criptográfica dos dados em trânsito.

2.2 Vulnerabilidade: *OS Command Injection*

O segundo achado selecionado foi identificado no arquivo `brokencrystals/src/app.service.ts` e corresponde a uma possível *OS Command Injection*, classificada pela ferramenta como *High*. A vulnerabilidade está associada ao *CWE-78 – Improper Neutralization of Special Elements used in an OS Command* e à categoria *A05:2025 – Injection do OWASP Top 10*. O problema ocorre quando dados controláveis externamente são utilizados para determinar comandos executados pelo sistema operacional sem restrições adequadas.

Na aplicação analisada, a rota `/api/spawn` recebe o parâmetro *command* diretamente de uma requisição HTTP e o encaminha ao método `launchCommand()`. Esse método divide o conteúdo recebido em um executável e seus argumentos e os fornece à função `spawn()`. Dessa forma, embora a chamada não utilize necessariamente um *shell* para interpretar metacaracteres, o próprio executável a ser iniciado pode ser definido pela entrada externa, caracterizando a possibilidade de execução arbitrária de comandos.

A falha enquadra-se em *A05:2025 – Injection* porque dados não confiáveis passam diretamente a um mecanismo de execução de comandos. Sua exploração pode comprometer a confidencialidade, integridade e disponibilidade do servidor, por exemplo por meio da leitura ou modificação de arquivos e execução de operações capazes de interromper o serviço.

Utilizando a ferramenta CVSS 4.0, foi obtido o vetor `CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`, resultando em *score* 9.3, correspondente à criticidade *Critical*. Em razão da possibilidade de execução remota de comandos sem autenticação, recomenda-se remediação imediata, com prazo máximo de 72 horas. Como medidas corretivas, recomenda-se eliminar a execução direta de comandos provenientes da entrada do usuário, utilizar APIs seguras sempre que possível e, quando a execução de processos externos for indispensável, implementar uma lista restritiva de executáveis e argumentos permitidos.

2.3 Resumo comparativo das vulnerabilidades

Tabela 1: Resumo das vulnerabilidades analisadas

Vulnerabilidade	CWE	OWASP 2025	CVSS 4.0	Criticidade	Remediação sugerida
<i>Unencrypted HTTP</i>	<i>CWE-319</i>	<i>A04:Cryptographic Failures</i>	7.1	High	15 dias
<i>OS Command Injection</i>	<i>CWE-78</i>	<i>A05:Injection</i>	9.3	Critical	72 horas

3 Análise SCA - OSV Scanner

Nesta seção são apresentadas as respostas às questões propostas, com ênfase na realização das análises no *OSV Scanner*. As capturas de tela utilizadas para documentar as etapas e os resultados obtidos encontram-se disponíveis no Apêndice B, ao final deste relatório.

3.1 CVE-2025-7783: *Form-data*

O *OSV Scanner* identificou no componente *form-data*, versão 2.3.3, a vulnerabilidade CVE-2025-7783, classificada como *Critical*, com *score* CVSS 4.0 de 9.4. A vulnerabilidade decorre do uso da função *Math.random()* para geração do *boundary* empregado em requisições `multipart/form-data`. Como os valores produzidos por essa função não são adequados para contextos em que a imprevisibilidade possui relevância para segurança, um atacante que consiga observar valores produzidos pelo mesmo gerador pode, sob determinadas condições, prever *boundaries* futuros. Com isso, torna-se possível inserir um *boundary* válido dentro de dados controlados pelo atacante, possibilitando *HTTP Parameter Pollution* e a inclusão de parâmetros adicionais na requisição.

O CVE está associado ao *CWE-330: Use of Insufficiently Random Values*, que representa o emprego de valores com aleatoriedade insuficiente em contextos de segurança dependentes de imprevisibilidade. Esse *CWE* é diretamente relacionado pela *MITRE* à categoria *A04:2025 - Cryptographic Failures do OWASP Top 10 2025*, razão pela qual a vulnerabilidade foi classificada nessa categoria. A correção oficial substituiu o uso de *Math.random()* por *crypto.randomBytes()*, utilizando uma fonte de aleatoriedade criptograficamente segura para gerar o *boundary*. Para a linha 2.x.x, a vulnerabilidade encontra-se corrigida a partir da versão 2.5.4. Dessa forma, recomenda-se atualizar o componente para uma versão corrigida e, caso isso não seja imediatamente possível, aplicar a mesma técnica de geração criptograficamente segura como mitigação temporária. Considerando o *score* CVSS 9.4 — *Critical*, a possibilidade

de elevado impacto sobre confidencialidade e integridade e a existência de prova de conceito pública, sugere-se um prazo máximo de 72 horas para remediação.

3.2 CVE-2026-3635: *Fastify*

O segundo *CVE* selecionado foi o *CVE-2026-3635*, identificado no componente *fastify*. A vulnerabilidade afeta versões até 5.8.2 e recebeu *score CVSS 3.1* de 6.1, correspondente à severidade *Medium*. O problema ocorre quando *trustProxy* é configurado com uma política restritiva de confiança, mas os valores disponibilizados por *request.protocol* e *request.host* ainda podem ser derivados dos cabeçalhos *X-Forwarded-Proto* e *X-Forwarded-Host* enviados por conexões não confiáveis. Dessa forma, um atacante que consiga se conectar diretamente ao servidor *Fastify*, contornando o *proxy* legítimo, pode falsificar o protocolo e o *host* percebidos pela aplicação. Isso é relevante caso tais informações sejam utilizadas para imposição de *HTTPS*, definição de *cookies* seguros, verificação de origem para *CSRF*, construção de *URLs* ou roteamento baseado no *host*.

O *CVE* está associado ao *CWE-348: Use of Less Trusted Source*, caracterizado pela utilização de uma fonte de informação menos confiável ou menos resistente a ataques quando deveria existir uma fonte mais confiável. Como o *CWE-348* não possui correspondência direta específica no *OWASP Top 10 2025*, a vulnerabilidade foi classificada neste trabalho como *A06:2025 — Insecure Design*, pois sua causa está relacionada à implementação inadequada de uma fronteira de confiança e à utilização de informações não confiáveis em decisões potencialmente sensíveis. Essa classificação é reforçada pela presença, em *A06*, de fraquezas conceitualmente próximas, como *Trust Boundary Violation* e *Reliance on Untrusted Inputs in a Security Decision*.

A correção oficial está disponível no *Fastify* 5.8.3, devendo o componente ser atualizado para essa versão ou superior. O *advisory* não apresenta *workaround* específico além da versão corrigida; como mitigação temporária, recomenda-se impedir conexões externas diretas ao *backend Fastify*, permitindo acesso apenas por meio do *proxy* confiável, além de não utilizar valores derivados de *X-Forwarded-** para decisões de segurança sem validação explícita. Considerando o *score CVSS 3.1: Medium* e as condições adicionais necessárias à exploração, sugere-se prazo máximo de 30 dias para remediação, reduzido quando a aplicação estiver diretamente acessível a redes não confiáveis.

4 Análise *DAST* - *Burp Suite*

4.1 Desafio e configuração do ambiente

Para a realização da terceira tarefa, foi escolhido o desafio “Basicamente Unidos”, da categoria *Web*, disponibilizado na plataforma *Forge RSI*. O objetivo da atividade consistiu em realizar a exploração manual de uma vulnerabilidade presente na aplicação com o auxílio do *Burp Suite*

Community Edition. Diferentemente da demonstração realizada em sala de aula, optou-se pela utilização do navegador integrado ao próprio *Burp Suite* (*Burp Browser*), por tornar mais prática a configuração do ambiente e a interceptação das requisições, dispensando a configuração manual de *proxy* e certificado em um navegador externo.

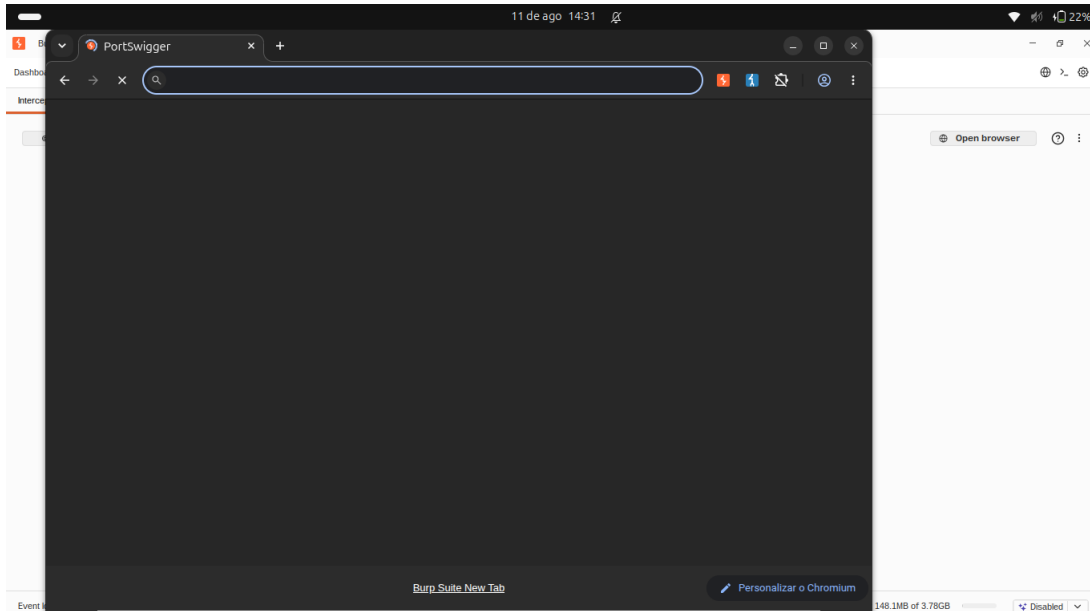


Figura 1: Tela do navegador interno do Burp Suite.

A partir do navegador integrado, foi acessada a plataforma *Forge RSI* e, entre os desafios disponíveis, selecionou-se o desafio “Basicamente Unidos”. Após iniciar a instância fornecida pela plataforma, foi possível acessar a aplicação vulnerável e iniciar a análise de sua funcionalidade de autenticação.

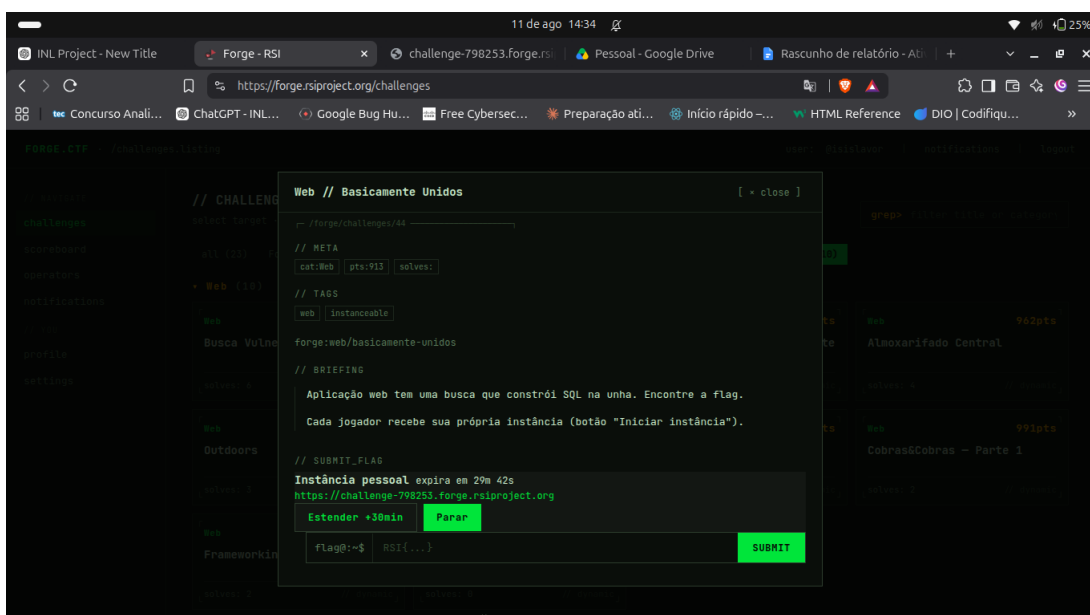


Figura 2: Desafio escolhido para realização da atividade.

4.2 Análise e exploração da vulnerabilidade

Inicialmente, foi analisado o formulário de autenticação apresentado pela aplicação. Com o auxílio do módulo *Proxy* do *Burp Suite*, foi possível observar a requisição HTTP enviada ao servidor e identificar os parâmetros utilizados para o envio das credenciais de usuário e senha. A análise indicou que os valores informados no formulário eram utilizados pelo servidor durante o processo de autenticação, tornando esses campos pontos relevantes para a realização dos testes.

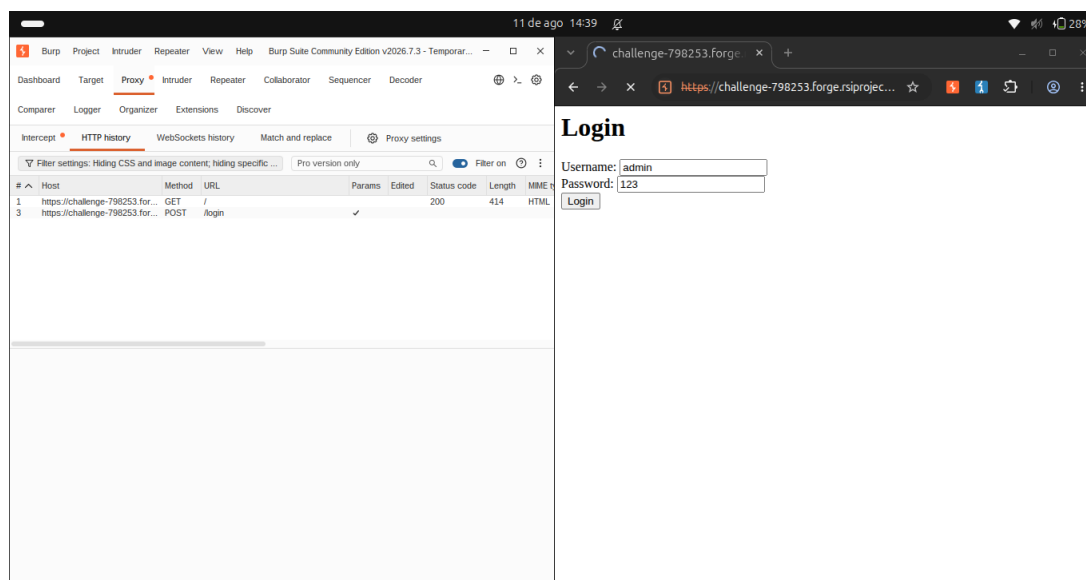


Figura 3: Tela inicial de proxy do Burp com visualização de requisições

Durante a etapa inicial de exploração, foram inseridos caracteres e expressões próprias da linguagem SQL no campo destinado ao nome de usuário, com o objetivo de verificar se a entrada fornecida pelo usuário era incorporada diretamente à consulta executada pelo banco de dados. Um dos testes realizados consistiu na substituição do nome de usuário por uma condição lógica sempre verdadeira, utilizando o *payload*: `admin' OR '1'='1`.

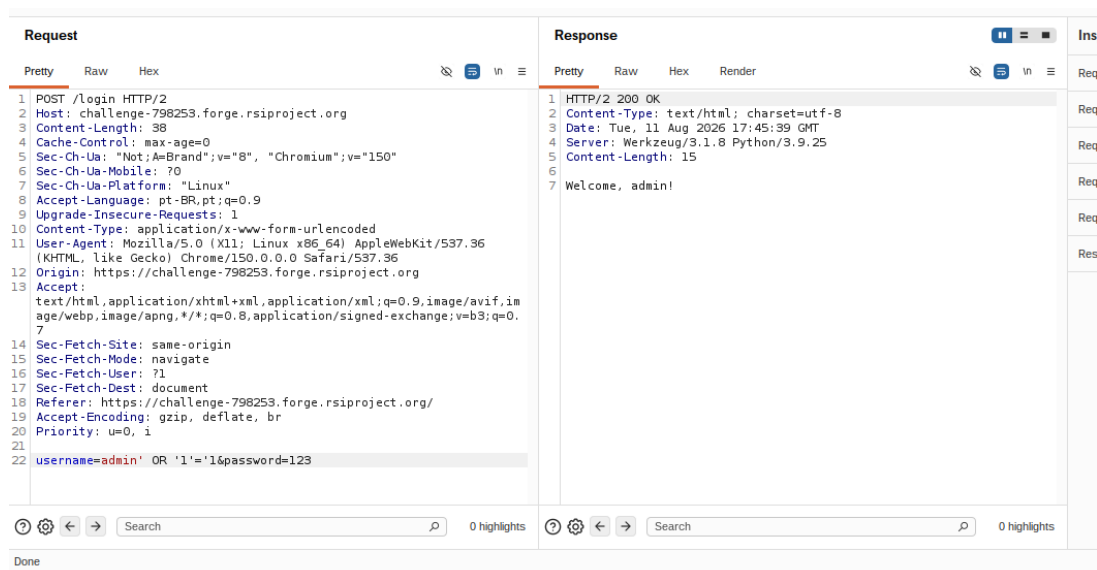


Figura 4: Primeira inserção de payload para teste.

Ao submeter a requisição modificada, a aplicação retornou a mensagem "Welcome, admin!", demonstrando que a condição inserida havia alterado a lógica originalmente utilizada na consulta de autenticação. Esse comportamento indicou que os dados fornecidos pelo usuário estavam sendo incorporados à consulta *SQL* sem tratamento adequado, confirmando a existência de uma vulnerabilidade de *SQL Injection*.

Após a confirmação da vulnerabilidade, passou-se à análise da estrutura da consulta *SQL* executada pela aplicação. Para isso, foram realizados testes utilizando o operador *UNION*, alterando gradativamente a quantidade de valores fornecidos na instrução. A finalidade dessa etapa era identificar quantas colunas eram retornadas pela consulta original, uma vez que uma operação *UNION SELECT* exige que as consultas combinadas apresentem uma quantidade compatível de colunas.

Os testes foram realizados aumentando progressivamente o número de valores fornecidos. Enquanto determinadas requisições eram aceitas pela aplicação, a tentativa com quatro colunas ocasionou uma mensagem de erro. A partir da comparação entre as respostas obtidas, foi possível determinar que a consulta original trabalhava com três colunas, informação necessária para construir as próximas cargas de exploração.

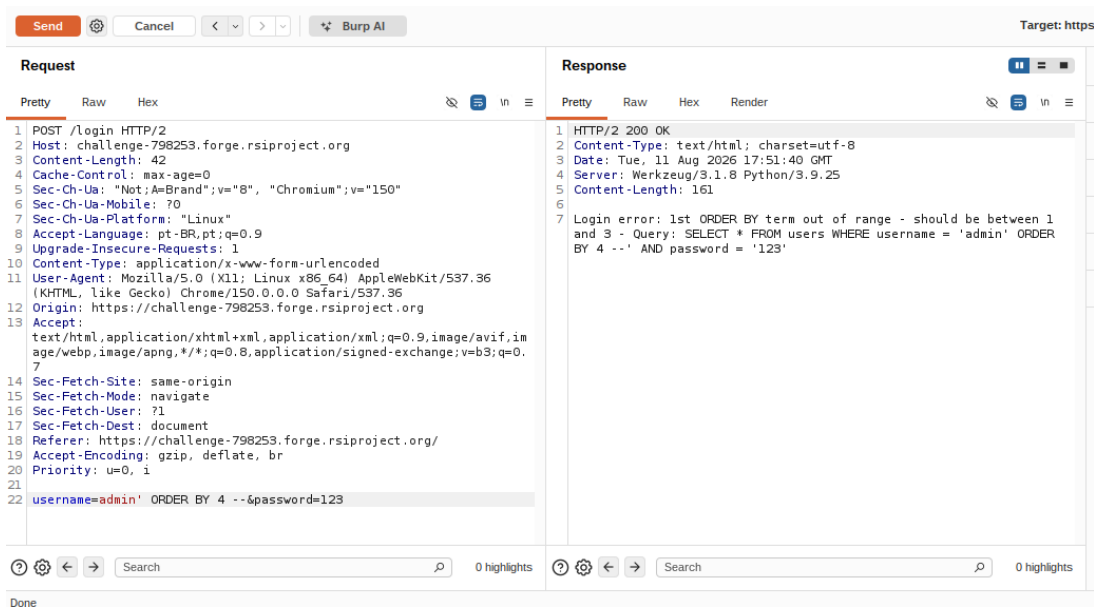


Figura 5: Visualização de orderby com tabela maior do que nosso valor desejado.

Após a confirmação da vulnerabilidade, passou-se à análise da estrutura da consulta *SQL* executada pela aplicação. Para isso, foram realizados testes utilizando o operador *UNION*, alterando gradativamente a quantidade de valores fornecidos na instrução. A finalidade dessa etapa era identificar quantas colunas eram retornadas pela consulta original, uma vez que uma operação *UNION SELECT* exige que as consultas combinadas apresentem uma quantidade compatível de colunas.

Os testes foram realizados aumentando progressivamente o número de valores fornecidos. Enquanto determinadas requisições eram aceitas pela aplicação, a tentativa com quatro colunas ocasionou uma mensagem de erro. A partir da comparação entre as respostas obtidas, foi possível determinar que a consulta original trabalhava com três colunas, informação necessária para construir as próximas cargas de exploração.

4.3 Evidência e obtenção de flag

Conhecendo a quantidade de colunas, tornou-se possível utilizar uma técnica de *UNION-Based SQL Injection*, na qual uma segunda consulta é combinada à consulta originalmente executada pela aplicação. Durante os testes, também foram analisadas as respostas retornadas pelo servidor para identificar informações existentes na base de dados. A aplicação utilizava *SQLite*, permitindo a consulta à tabela interna `sqlite_master`, que mantém informações referentes à estrutura do banco de dados, como tabelas e comandos empregados em sua criação. Para analisar essa estrutura, foi utilizada uma consulta baseada em *UNION SELECT*, compatível com as três colunas anteriormente identificadas. Um dos *payloads* empregados durante essa etapa foi: `x' UNION SELECT 1, sql, 3 FROM sqlite_master --`. A resposta produzida pela aplicação permitiu obter informações sobre a estrutura interna da base de dados, contribu-

indo para a identificação dos objetos que poderiam armazenar os dados relevantes ao desafio.

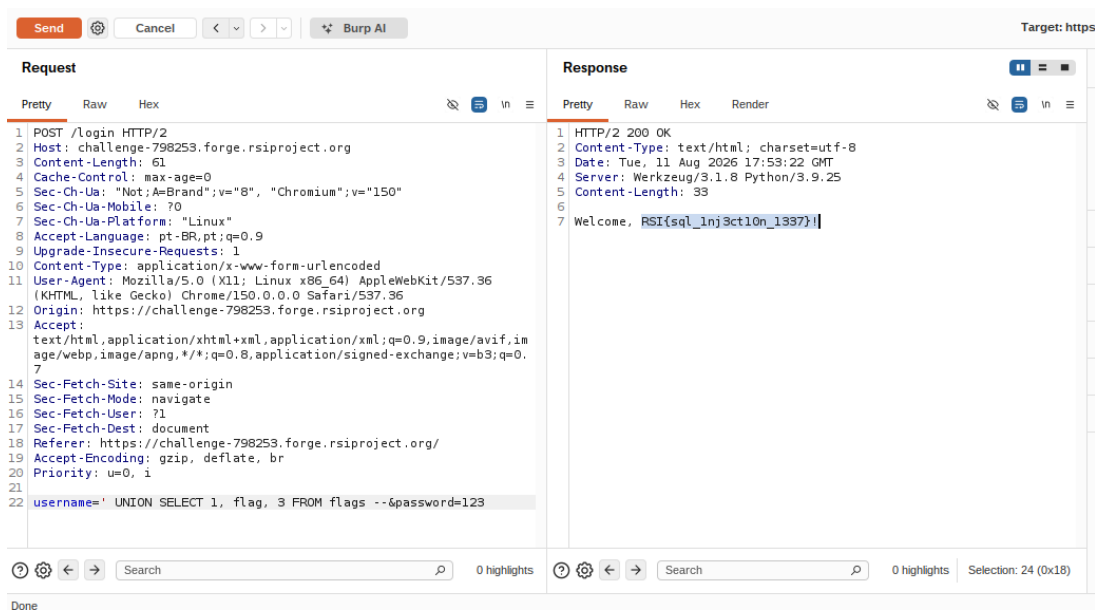


Figura 6: Payload que gera a resolução final do desafio.

A análise das respostas anteriores também havia revelado referências ao valor “*Flag*”, indicando a existência de informação relacionada à *flag* procurada. A partir das informações obtidas sobre a estrutura do banco, foi identificada a tabela *flags* e o campo *flag*. Dessa forma, foi elaborada uma nova consulta utilizando novamente a técnica *UNION SELECT*, desta vez direcionada diretamente aos dados armazenados nessa tabela.

O *payload* utilizado foi: `' UNION SELECT 1, flag, 3 FROM flags --`. A requisição foi então enviada ao servidor pelo *Burp Repeater*, permitindo modificar e repetir manualmente as requisições sem a necessidade de reenviar o formulário pelo navegador a cada tentativa. Como resposta, o servidor retornou *HTTP 200 OK* e apresentou no conteúdo da página o valor armazenado no campo consultado. Com a exploração, foi possível recuperar a *flag* correspondente ao desafio: `RSI{sql_1nj3ct10n_1337}`.

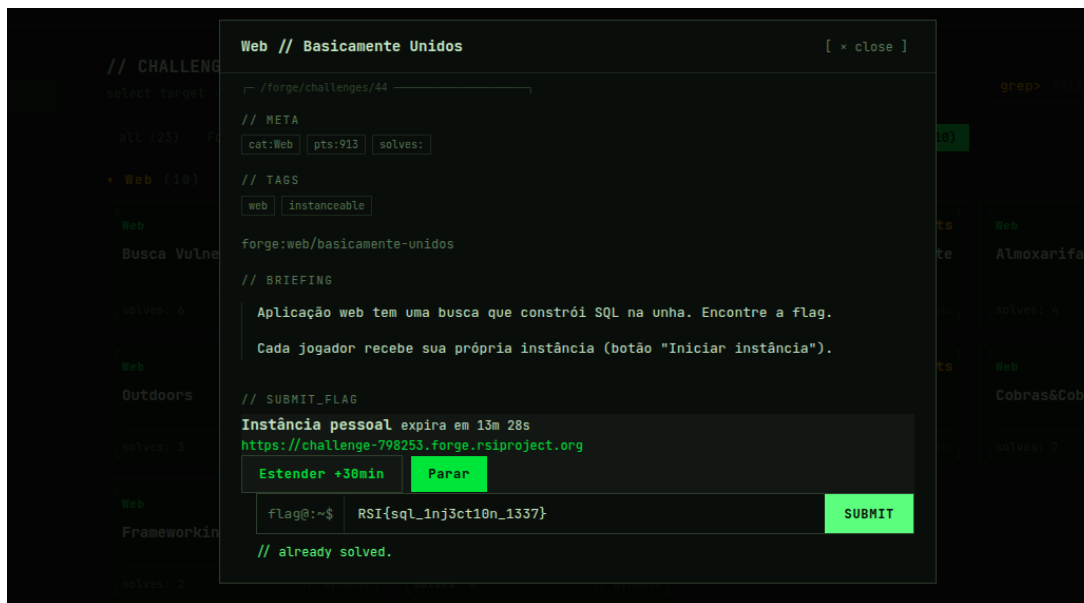


Figura 7: Resolução final do desafio na plataforma.

4.4 Recomendações sobre a vulnerabilidade

Dessa forma, o desafio “Basicamente Unidos” foi concluído com sucesso por meio da exploração de uma vulnerabilidade de *SQL Injection*, desde a identificação da falha na autenticação até a análise da estrutura do banco de dados e a recuperação da informação armazenada. O desafio também havia sido resolvido anteriormente sem o auxílio do *Burp Suite* como forma de treinamento; contudo, para esta atividade, todo o procedimento foi reproduzido utilizando a ferramenta, especialmente os módulos *Proxy*, *Burp Browser* e *Repeater*, conforme exigido na proposta da tarefa.

A vulnerabilidade identificada decorre principalmente da utilização direta dos valores fornecidos pelo usuário na construção de comandos *SQL*. Como medida de remediação, a aplicação deve evitar a concatenação de *strings* para montar consultas ao banco de dados e utilizar consultas parametrizadas (*prepared statements*). Com essa abordagem, os valores fornecidos pelo usuário são tratados exclusivamente como dados, e não como parte da sintaxe *SQL*, impedindo que caracteres especiais ou comandos inseridos nos campos da aplicação alterem a estrutura da consulta executada. Recomenda-se, adicionalmente, a aplicação de validação adequada das entradas e a adoção do princípio do menor privilégio para as contas utilizadas no acesso ao banco de dados.

5 Conclusão

A realização das atividades permitiu aplicar, de forma prática, diferentes abordagens utilizadas na identificação e análise de vulnerabilidades em aplicações web. O uso das ferramen-

tas *Semgrep*, *OSV Scanner* e *Burp Suite* possibilitou observar vulnerabilidades sob diferentes perspectivas, abrangendo tanto o código-fonte e suas dependências quanto o comportamento da aplicação durante sua execução. Além da identificação das falhas, a análise das vulnerabilidades permitiu compreender melhor seus impactos, níveis de criticidade e possíveis formas de remediação. Dessa forma, a atividade reforçou a importância da utilização conjunta de técnicas *SAST*, *SCA* e *DAST* ao longo do desenvolvimento de software, contribuindo para uma avaliação de segurança mais completa e para a adoção de medidas preventivas e corretivas adequadas.

Referências

- [1] FIRST. CVSS v4.0 Calculator. <https://www.first.org/cvss/calculator/4.0>, 2023. Acesso em: 12 ago. 2026.
- [2] FIRST. CVSS v4.0 Specification Document. <https://www.first.org/cvss/specification-document>, 2023. Acesso em: 12 ago. 2026.
- [3] MITRE. CWE-319: Cleartext Transmission of Sensitive Information. <https://cwe.mitre.org/data/definitions/319>, 2025. Acesso em: 12 ago. 2026.
- [4] OWASP Foundation. A04:2025 – Cryptographic Failures. https://owasp.org/Top10/2025/A04_2025-Cryptographic_Failures/, 2025. Acesso em: 12 ago. 2026.
- [5] OWASP Foundation. OWASP Top 10:2025. <https://owasp.org/Top10/2025/>, 2025. Acesso em: 12 ago. 2026.

A Apêndice: SAST com Semgrep

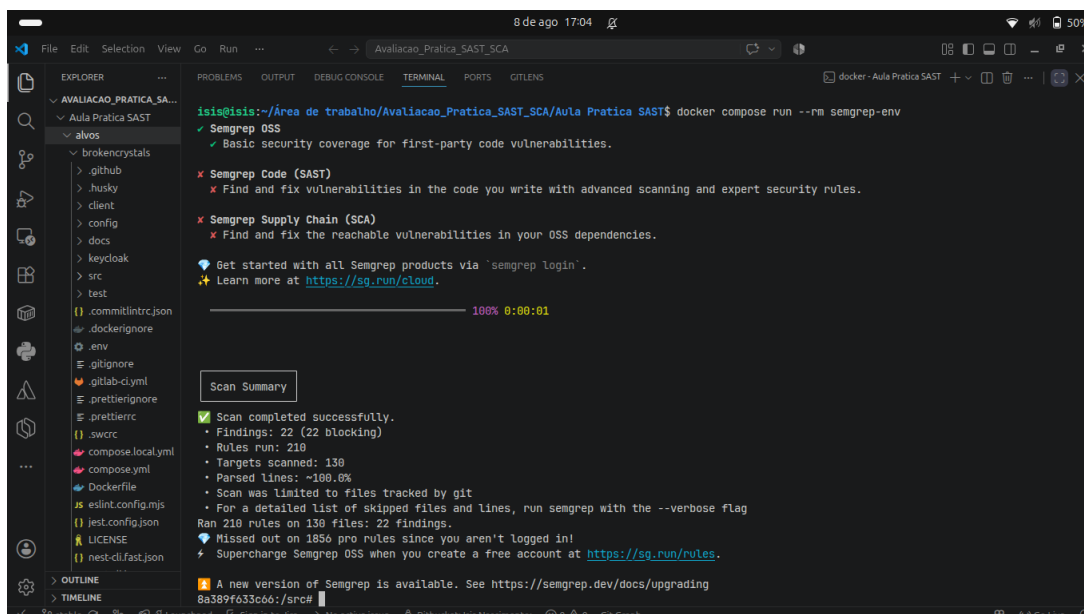


Figura 8: Terminal com comandos de análise semgrep.

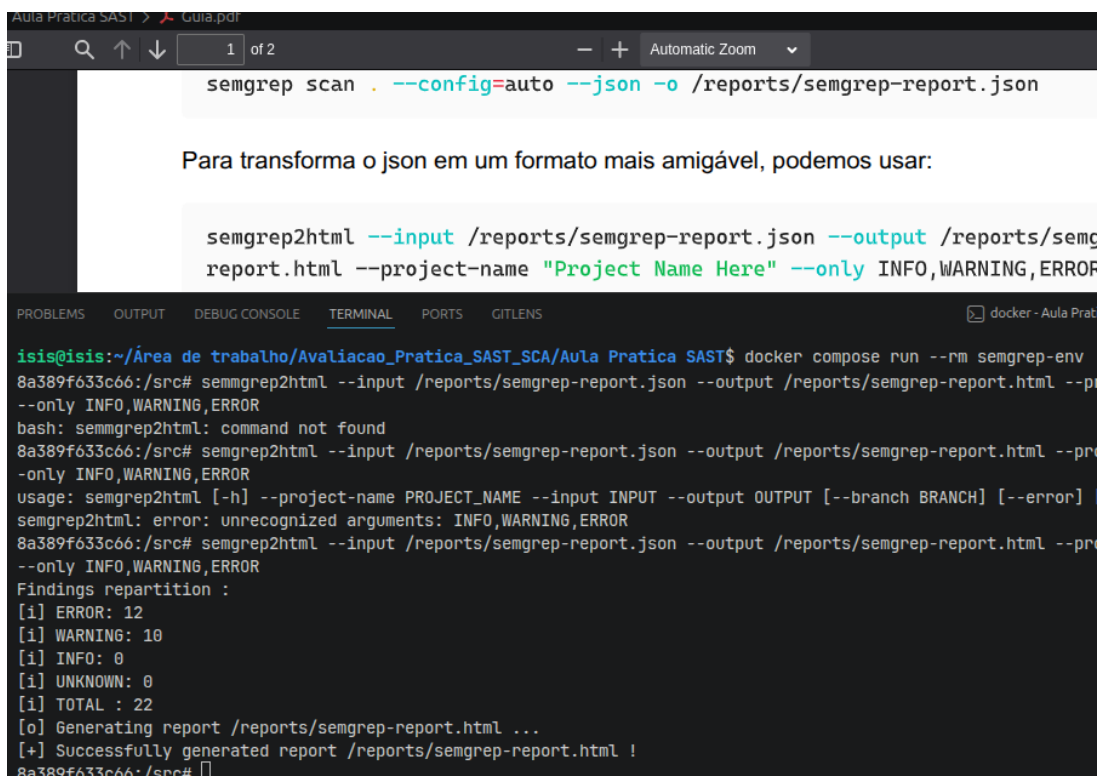
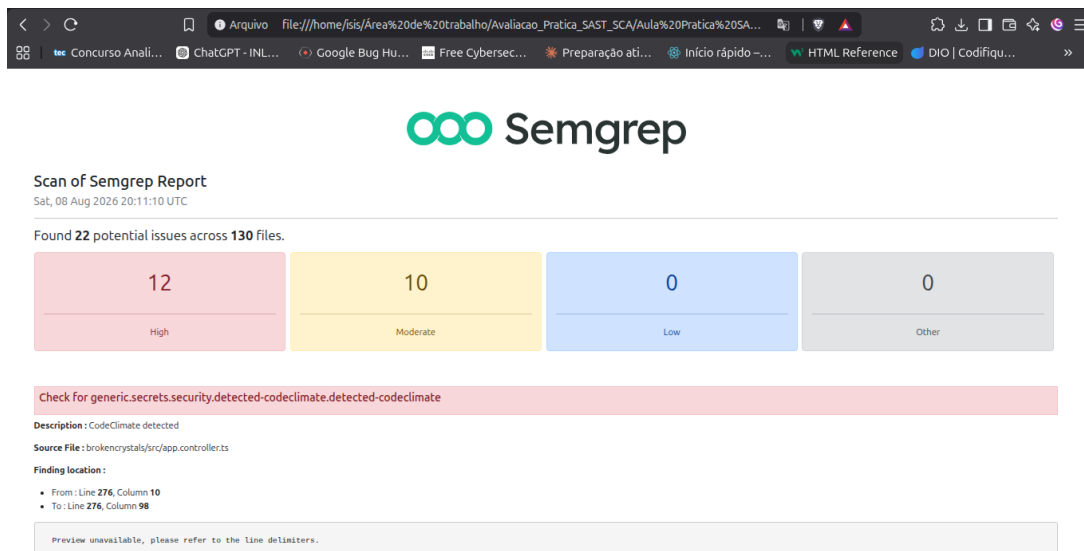


Figura 9: Terminal com comandos para transformar os resultados em formato HTML.



B Apêndice: SCA com OSV Scanner

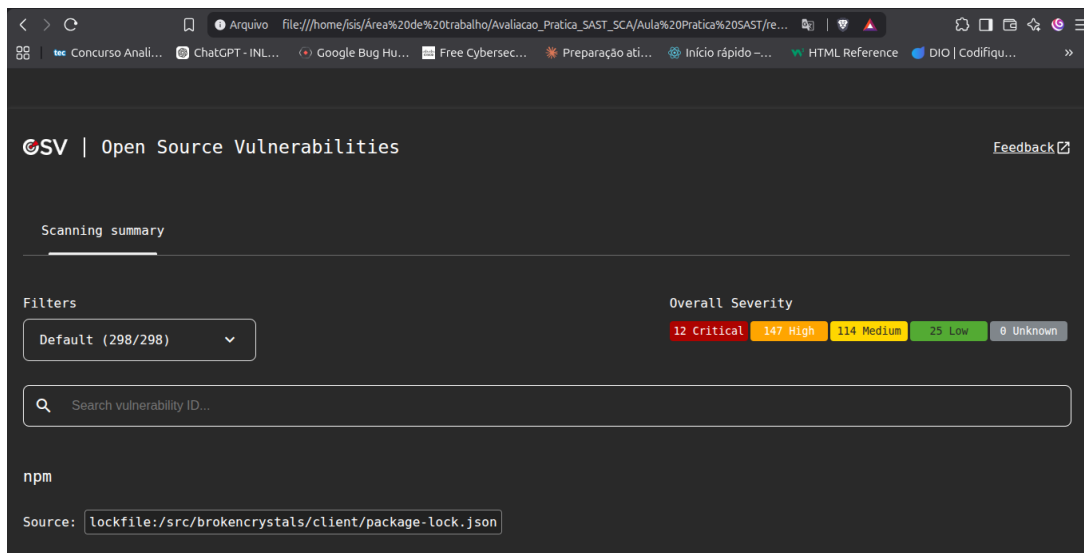


Figura 12: Resultado da análise.

***Observação:** Nesta versão temos 298/298 vulnerabilidades em todo o repositório, podem ser outras novas vulnerabilidades de versões mais recentes ou terem sido adicionadas posteriormente, mas foi escaneado corretamente de acordo com as especificações e caminho dos arquivos.